Programação Funcional

(COMP0393)

Leila M. A. Silva

Classes de Tipos
Checagem de Tipos
(COMP0393)

Aula 13

Classes de Tipos

- Uma classe de tipos é uma coleção de tipos caracterizada por um conjunto de "nomes" de funções/operações com seus tipos.
- *Equality* class, `Eq`: compreende os elementos que podem ser comparados por igualdade e desigualdade.

```
class Eq a where
  (==), (/=) :: a -> a -> Bool
```

assinatura da classe

- As classes possuem instâncias. Cada instância (um tipo) da classe implementa suas próprias funções/operações.

```
instance Eq Bool where
   (==) :: a -> a -> Bool
   True == True = True
   False == False = True
   _ == _ = False
   (/=) :: a -> a -> Bool
    x /= y = not (x == y)
```

Classes de Tipos

- A declaração das classes podem incluir definições *default*. Neste caso as instâncias da classe não precisam redefinir estas definições *default*. Por exemplo, as instâncias da classe `Eq` abaixo só têm a obrigação de dar uma definição para (==); redefinir (/=) é opcional.

```
class Eq a where
 (==), (/=) :: a -> a -> Bool
  x /= y = not (x==y)
```

Classes de Tipos

- Também é possível fazer

```
class Eq a where
 (==), (/=) :: a -> a -> Bool
  x == y == not (x /=y)
  x /= y = not (x==y)
```

- Neste caso as instâncias têm a obrigação de definir apenas um dos operadores (==) ou (/=); a definição de ambos é opcional.

Instâncias com Contexto

- Pares do tipo `(a, b)` podem ser comparados por `(==)` caso `a` e `b` sejam instâncias de `Eq`

```
instance (Eq a, Eq b) => Eq (a, b) where
   (==) :: (a,b) -> (a,b) -> Bool
   (x1, x2) == (y1, y2) = x1 == y1 && x2 == y2
```

Exemplo de Função que Usa Eq

- A função abaixo que verifica se três inteiros são iguais, na realidade não tem nada específico para inteiros. Portanto, poderia ser aplicada a qualquer elemento que pudesse ser comparado com `(==)`, ou seja a qualquer elemento cujo tipo pertencesse à classe `Eq`.

```
todosIguais :: Int -> Int -> Int -> Bool
todosIguais m n p = (m==n) && (n==p)
```

```
todosIguais :: Eq a => a-> a -> a -> Bool
todosIguais m n p = (m==n) && (n==p)
```

```
todosIguais :: Char -> Char -> Char -> Bool
todosIguais :: (Int,Bool) -> (Int,Bool) -> (Int,Bool)->Bool
```

```
todosIguas 'a' 'b' 'a'
todosIguais (2,True) (3,False)(4,True)
todosIguais succ succ succ
```

Erro!!! Funções não pertencem à classe Eq!!

Classes Derivadas

- Uma classe pode depender de outra. Podemos interpretar que a classe `Ord` **herda** a assinatura da classe `Eq`

```
class Eq a => Ord a where
    (<), (<=), (>), (>=) :: a -> a -> Bool
    max, min :: a -> a -> a
    compare :: a -> a -> Ordering
```

- Para um tipo pertencer à classe `Ord`, é necessário prover definições para as operações `Eq`, além das definições para as funções/operadores da classe `Ord`.
- `Ordering` é um tipo de dados que assume os valores `LT` (*lesser then*), `EQ` (*equal*) e GT (*greater then*) – veremos em detalhes mais adiante

Classes Derivadas

- Exemplo de funçao definida sobre tipos da classe `Ord`. `ordInsercao`

```
insOrd :: Ord a => a -> [a] -> [a]
insOrd y [] = [y]
insOrd y (z:zs)
  | y <= z = y : z : zs
  | otherwise = z: insOrd y zs

ordInsercao :: Ord a => [a] -> [a]
ordInsercao [] = []
ordInsercao (x:xs) = insOrd x (ordInsercao xs)
```

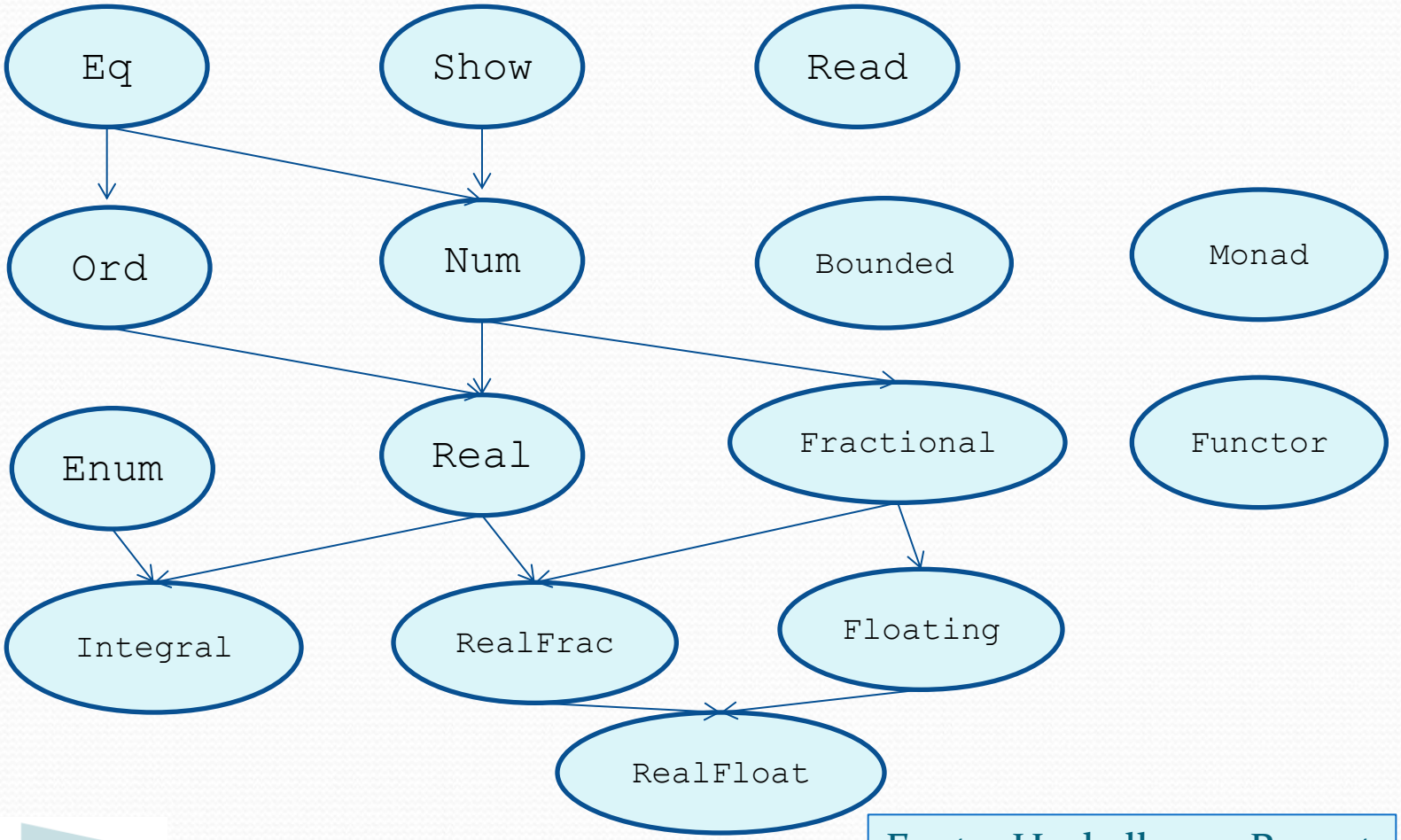
- Desta forma qualquer elemento da classe `Ord` pode ser ordenado por `ordInsercao`. Fica a cargo da definição das instâncias das classes `Ord` dizer como estes elementos serão comparados e `ordInsercao` se mantém inalterada.

Exercícios Recomendados

- Quais os tipos mais gerais das funções:
  - `max`
  - `maxTres`
  - `quickSort`
  - `ordSelecao`
  - `elem`
- Defina o tipo listas como instância de `Eq`.

Principais Classes de Tipos

- Além da classe Eq e Ord, Haskell provê várias classes pré-definidas:.



Fonte: Haskell 2010 Report

Principais Classes de Tipos

Classe	Tipos que são instâncias da classe
`Eq, Show, Read`	Todos, à exceção de `IO` e funções
`Ord`	Todos, à exceção de `IO, IOError` e funções
`Num`	`Int, Integer, Float, Double`
`Bounded`	`Int, Char, (), Bool, Ordering` e tuplas
`Enum`	`Bool, Char, (), Float, Double, Int, Integer`
`Real`	`Int, Integer, Double, Float`
`Fractional`	`Float, Double`
`Integral`	`Int, Integer`
`RealFrac`	`Float, Double`
`Floating`	`Float, Double`
`RealFloat`	`Float, Double`
`Monad,Functor`	`IO, Maybe, []`

Visão Geral das Classes

- `Eq` e `Ord` – já vistas
- `Show`
  - Inclui tipos cujos valores podem ser escritos como strings

```
type ShowS = String -> String
class Show a where
  showsPrec:: Int -> a -> ShowS
  show :: a -> String
  showList :: [a] -> ShowS
```

```
instance Show Bool where
  show True = "True"
  show False = "False"
```

Visão Geral das Classes

- `Read`
  - Inclui tipos cujos valores podem ser lidos de strings
  - Função principal:

```
read :: (Read a) => String -> a
```

- `Bounded`
  - Fornece o menor e o maior valor dos tipos que são instâncias da classe

```
class Bounded a where
  maxBound, minBound :: a
```

Visão Geral das Classes

- `Enum`
  - Útil para gerar listas usando expressões de enumeração. Por exemplo: [2,4..10].
  - A enumeração atua não só nos inteiros mas em todos os tipos que são instâncias da classe.

```
class Enum a where
  succ, pred :: a -> a
  toEnum :: Int -> a      -- conversão Int e o tipo
  fromEnum :: a -> Int    -- conversão tipo e Int
  enumFrom :: a -> [a]    -- [n ..]
  enumFromThen :: a -> a -> [a] -- [n,m..]
  enumFromTo :: a -> a -> [a]  -- [n..m]
  enumFromThenTo :: a -> a -> a -> [a] - [n,p..m]
```

Visão Geral das Classes

- Num
  - Classe base para as classes numéricas de Haskell.

```
class (Eq a, Show a) => Num a where
  (+), (-), (*) :: a -> a -> a
  negate :: a -> a
  abs, signum :: a -> a
  fromInteger :: Integer -> a

  x-y = x + negate y
```

Visão Geral das Classes

- Outras classes numéricas herdam de `Num` e acrescentam mais funções. Ex:

```
class (Num a) => Fractional a where
  (/):: a -> a -> a
  recip :: a -> a
  fromRational :: Rational -> a

  recip x  = 1 / x
```

```
class (Fractional a) => Floating a where
  pi:: a
  exp, log, sqrt :: a -> a
 (**), logBase :: a -> a -> a
  sin, cos, tan :: a -> a
  ...
```

Visão Geral das Classes

- `Integral`: Inclui somente números inteiros, que podem inclusive ser números reais que sejam inteiros.

```
class (Real a, Enum a) => Integral a where
  quot, rem, div, mod :: a -> a -> a
  quotRem, divMod :: a -> a -> (a,a)
  even, odd :: a -> Bool
  ...
```

Checagem de Tipos

- Funções Monomórficas

```
f :: t1 -> t2 -> ... -> tk -> tr

f p1 p2 ... Pk
  | g1 = e1
  | g2 = e2
  | ...
  | gk = ek
```

- Precisamos checar:
  - O tipo resultante de cada guarda `gi`, é `Bool`;
  - O valor retornado em todas as expressões `ei` é do tipo `tr`;
  - O padrão `pj` tem que ser consistente com o tipo de `tj` .

Checagem de Tipos

- Funções Polimórficas: os tipos são inferidos levando-se em conta restrições que precisam ser satisfeitas

```
f (x,y) = (x, ['a' .. y])
```

```
f :: (a, [Char]) -> (a,[Char])
```

```
g (m,zs) = m + length zs
```

```
g :: (Int, [a]) -> Int
```

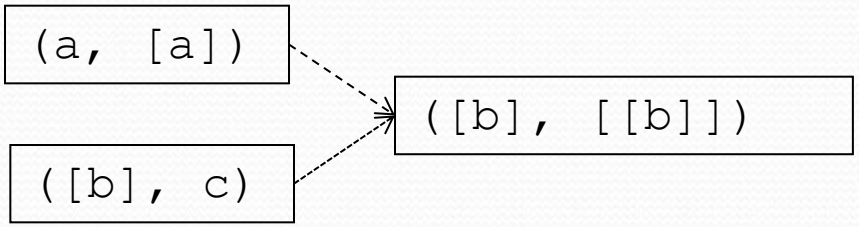
```
h = g.f
g :: (Int, [b]) -> Int
f :: (a, Char) -> (a,[Char])
```

```
h :: (Int, Char) -> Int
```

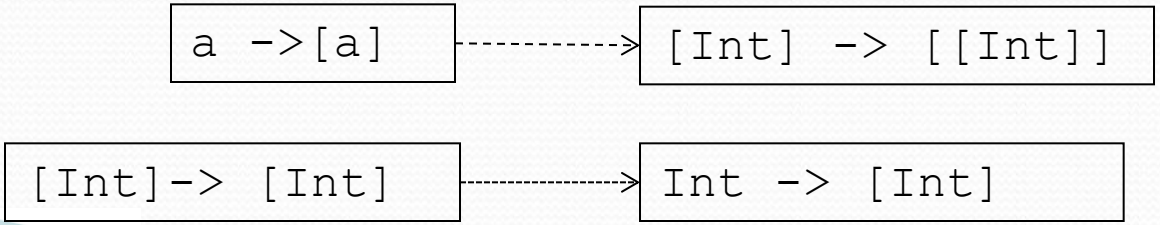
- Neste último exemplo dizemos que estamos **unificando** `(a,[Char])` com `(Int, [b])`, ou seja, tentando estabelecer a instância comum mais geral das duas expressões de tipos.

Checagem de Tipos

- A unificação pode gerar politipos. Ex: Suponha que desejamos unificar `(a, [a])` e `([b],c)`



- Mas nem sempre é possível unificar... Neste caso há conflito de tipo



Incompatível!!!

Checagem de Tipos

- Checagem de funções seguem as mesmas regras de consistências de tipos que as funções monomórficas:
  - O tipo resultante de cada guarda `gi`, é `Bool`;
  - O valor retornado em todas as expressões `ei` é consistente com o tipo `tr`;
  - O padrão `pj` tem que ser consistente com o tipo de `tj` .

Exercícios Recomendados

- Diga se é possível unificar os seguintes tipo e se for diga qual o tipo mais geral resultante da unificação:
  - `(Int-> b)  (a->Bool)`
  - `(Int, a, a)  (a, a, [Bool])`
- A função `f :: (a,[a]) -> b` pode ser aplicada com os argumentos `(2, [3])`, `(2,[])` e `(2 [True])`? Justifique sua resposta.
- E se fosse `f :: (a,[a]) -> a` na questão acima?